

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

**MODUL XIV
GRAPH**



Disusun Oleh :

NAMA : Damanik, Yohanes Geovan Ondova
NIM : 103112400022

Dosen

FAHRUDIN MUKTI WIBOWO

**PROGRAM STUDI STRUKTUR DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025**

A. Dasar Teori

Graph: Himpunan tidak kosong dari *node* (atau *vertex*) dan garis penghubung (*edge*).

Jenis jenis Graph:

- Graph Berarah (Directed Graph): Merupakan *graph* di mana setiap *node* memiliki *edge* yang memiliki arah ke mana *node* tersebut dihubungkan.
- Graph Tidak Berarah (Undirected Graph): Merupakan *graph* di mana setiap *node* memiliki *edge* yang menghubungkan ke *node* lain tanpa arah.

Topological Sort:

- Pengertian: Topological Sort bertujuan untuk mendapatkan keterurutan linear dari elemen suatu himpunan yang memiliki urutan parsial (sebagian).
- Proses: Proses untuk mendapatkan urutan linear dilakukan dengan memilih item yang tidak memiliki *predecessor* (jumlah *predecessor* = nol), menghapus item tersebut dari himpunan S dan memasukkannya ke dalam *list* hasil, dan mengulangnya sampai himpunan S kosong.
- Solusi 1 (Multi List): Menggunakan representasi **Multilist** dengan dua jenis *list* berkait:
 1. *List "Leader"* (horizontal) yang mencatat banyaknya *predecessor* setiap item.
 2. *List "Trailer"* (vertikal) yang merupakan *indirect addressing* ke setiap successor
- Solusi 2 (List Linear Sederhana): Menerjemahkan proses secara fungsional dengan representasi *list* linear sederhana yang terdiri dari elemen

Guided (berisi screenshot source code & output program disertai penjelasannya)

Guided 1

graf.cpp

```
#include "graf.h"
#include <queue>
#include <stack>

void createGraph(Graph &G) {
    G.first = NULL;
}

addrNode AllocateNode(addrNode X) {
    addrNode P = new ElmNode;
    P->info = X;
    P->visited = 0;
    P->firstEdge = NULL;
    P->next = NULL;
    return P;
}

addrEdge AllocateEdge(addrNode N) {
    addrEdge P = new ElmEdge;
    P->node = N;
    P->next = NULL;
    return P;
}

void InsertNode(Graph &G, addrNode X) {
    addrNode P = AllocateNode(X);
    P->next = G.first;
    G.first = P;
}

addrNode FindNode(Graph G, addrNode X) {
    addrNode P = G.first;
    while (P != NULL) {
        if (P->info == X)
            return P;
        P = P->next;
    }
    return NULL;
}

void ConnectNode(Graph &G, addrNode A, addrNode B) {
    addrNode N1 = FindNode(G, A);
    addrNode N2 = FindNode(G, B);

    if (N1 == NULL || N2 == NULL) {
```

```

        cout << "Node tidak ditemukan\n";
        return;
    }

    // Buat Edge dari N1 ke N2
    addrEdge E1 = AllocateEdge(N2);
    E1->next = N1->firstEdge;
    N1->firstEdge = E1;

    // Karena undirected -> buat Edge balik
    addrEdge E2 = AllocateEdge(N1);
    E2->next = N2->firstEdge;
    N2->firstEdge = E2;
}

void PrintInfoGraph(Graph G) {
    addrNode P = G.first;
    while (P != NULL) {
        cout << P->info << "-> ";
        addrEdge E = P->firstEdge;
        while (E != NULL) {
            cout << E->node->info << " ";
            E = E->next;
        }
        cout << endl;
        P = P->next;
    }
}

void ResetVisited(Graph &G) {
    addrNode P = G.first;
    while (P != NULL) {
        P->visited = 0;
        P = P->next;
    }
}

void PrintDFS(Graph G, addrNode N) {
    if (N == NULL)
        return;
    N->visited = 1;
    cout << N->info << " ";

    addrEdge E = N->firstEdge;
    while (E != NULL)
    {
        if (E->node->visited == 0) {
            PrintDFS(G, E->node);
        }
        E = E->next;
    }
}

```

```

    }
}

void PrintBFS(Graph G, addrNode N) {
    if (N == NULL)
        return;

    queue<addrNode> Q;
    Q.push(N);

    while (!Q.empty()) {
        addrNode curr = Q.front();
        Q.pop();
        if (curr->visited == 0) {
            curr->visited = 1;
            cout << curr->info << " ";

            addrEdge E = curr->firstEdge;
            while (E != NULL) {
                if (E->node->visited == 0) {
                    Q.push(E->node);
                }
                E = E->next;
            }
        }
    }
}

```

graf.h

```

#ifndef GRAF_H_INCLUDED
#define GRAF_H_INCLUDED

#include <iostream>
using namespace std;

typedef char infograf;

struct ElmNode;
struct ElmEdge;

typedef ElmNode* addrNode;
typedef ElmEdge* addrEdge;

struct ElmNode {
    infograf info;
    int visited;
    addrEdge firstEdge;
}

```

```

    addrNode next;
};

struct ElmEdge {
    addrNode node;
    addrEdge next;
};

struct Graph {
    addrNode first;
};

// PRIMITIF GRAPH
void createGraph(Graph &G);
addrNode AllocateNode(intograf X);
addrEdge AllocateEdge(addrNode N);

void InsertNode(Graph &G, intograf X);
addrNode FindNode(Graph G, intograf X);

void ConnectNode(Graph &G, intograf A, intograf B);

void PrintGraph(Graph G);

// Traversal
void ResetVisited(Graph &G);
void PrintDFS(Graph G, addrNode N);
void PrintBFS(Graph G, addrNode N);

#endif

```

main.cpp

```

#include "graf.h"
#include "graf.cpp"
#include <iostream>
using namespace std;

int main(){

    Graph G;
    createGraph(G);

    InsertNode(G, 'A');
    InsertNode(G, 'B');
    InsertNode(G, 'C');
    InsertNode(G, 'D');
}

```

```

    ConnectNode(G, 'A', 'B');
    ConnectNode(G, 'A', 'C');
    ConnectNode(G, 'B', 'D');
    ConnectNode(G, 'C', 'Ex');

    cout << "=== Struktur Graph ===\n";
    PrintInfoGraph(G);

    cout << "\n===DFS dari node A===\n";
    ResetVisited(G);
    PrintDFS(G, FindNode(G, 'A'));

    cout << "\n===DFS dari node A===\n";
    ResetVisited(G);
    PrintDFS(G, FindNode(G, 'A'));

    cout<<endl;
    return 0;
}

```

Output

```

Node tidak ditemukan
=== Struktur Graph ===
D -> B
C -> A
B -> D A
A -> C B

===DFS dari node A===
A C B D
===DFS dari node A===
A C B D
PS C:\Users\Lenovo\Documents\STRUKTUR DATA MODUL 14>

```

Deskripsi:

Program ini mengimplementasikan struktur data graf tidak berarah (undirected graph) menggunakan adjacency list dalam bahasa C++, terdiri dari header file (graf.h) yang mendefinisikan tipe data node dan edge beserta primitif-operasinya, serta file implementasi (graf.cpp) yang berisi fungsi-fungsi untuk membuat graf, menambahkan node, menghubungkan node, serta melakukan traversal Depth-First Search (DFS) dan Breadth-First Search (BFS). Pada file main.cpp, program membuat graf dengan empat node ('A', 'B', 'C', 'D'), menghubungkannya sesuai pola tertentu, menampilkan struktur graf, kemudian melakukan traversal DFS mulai dari node 'A' sebanyak dua kali setelah mereset status kunjungan.

B. Unguided/Tugas (berisi screenshot source code & output program disertai penjelasannya)

Unguided 1

graph.cpp

```
#include "graph.h"
#include <queue>

void CreateGraph(Graph &G) {
    G.first = NULL;
}

adrNode AllocateNode(infoGraph X) {
    adrNode P = new ElmNode;
    P->info = X;
    P->visited = 0;
    P->firstEdge = NULL;
    P->next = NULL;
    return P;
}

adrEdge AllocateEdge(adrNode N) {
    adrEdge P = new ElmEdge;
    P->node = N;
    P->next = NULL;
    return P;
}

void InsertNode(Graph &G, infoGraph X) {
    adrNode P = AllocateNode(X);
    P->next = G.first;
    G.first = P;
}

adrNode FindNode(Graph G, infoGraph X) {
    adrNode P = G.first;
    while (P != NULL) {
        if (P->info == X)
            return P;
        P = P->next;
    }
    return NULL;
}

void ConnectNode(adrNode N1, adrNode N2) {
    if (N1 == NULL || N2 == NULL) {
        return;
    }

    // Edge dari N1 ke N2
```



```

    adrEdge E1 = AllocateEdge(N2);
    E1->next = N1->firstEdge;
    N1->firstEdge = E1;

    // Edge dari N2 ke N1 (undirected)
    adrEdge E2 = AllocateEdge(N1);
    E2->next = N2->firstEdge;
    N2->firstEdge = E2;
}

void PrintInfoGraph(Graph G) {
    adrNode P = G.first;
    while (P != NULL) {
        cout << P->info << " -> ";
        adrEdge E = P->firstEdge;
        while (E != NULL) {
            cout << E->node->info << " ";
            E = E->next;
        }
        cout << endl;
        P = P->next;
    }
}

void ResetVisited(Graph &G) {
    adrNode P = G.first;
    while (P != NULL) {
        P->visited = 0;
        P = P->next;
    }
}

// Soal 2: Prosedur PrintDFS
void PrintDFS(Graph G, adrNode N) {
    if (N == NULL)
        return;

    N->visited = 1;
    cout << N->info << " ";

    adrEdge E = N->firstEdge;
    while (E != NULL) {
        if (E->node->visited == 0) {
            PrintDFS(G, E->node);
        }
        E = E->next;
    }
}

// Soal 3: Prosedur PrintBFS

```

```

void PrintBFS(Graph G, adrNode N) {
    if (N == NULL)
        return;

    queue<adrNode> Q;
    Q.push(N);

    while (!Q.empty()) {
        adrNode curr = Q.front();
        Q.pop();

        if (curr->visited == 0) {
            curr->visited = 1;
            cout << curr->info << " ";

            adrEdge E = curr->firstEdge;
            while (E != NULL) {
                if (E->node->visited == 0) {
                    Q.push(E->node);
                }
                E = E->next;
            }
        }
    }
}

```

graph.h

```

#ifndef GRAPH_H_INCLUDED
#define GRAPH_H_INCLUDED

#include <iostream>
using namespace std;

typedef char infoGraph;

struct ElmNode;
struct ElmEdge;

typedef ElmNode* adrNode;
typedef ElmEdge* adrEdge;

struct ElmNode {
    infoGraph info;
    int visited;
    adrEdge firstEdge;
    adrNode next;
};

struct ElmEdge {

```

```

    adrNode node;
    adrEdge next;
};

struct Graph {
    adrNode first;
};

// Primitif sesuai soal latihan 1
void CreateGraph(Graph &G);
void InsertNode(Graph &G, infoGraph X);
void ConnectNode(adrNode N1, adrNode N2);
void PrintInfoGraph(Graph G);

// Primitif tambahan untuk mendukung
adrNode AllocateNode(infoGraph X);
adrEdge AllocateEdge(adrNode N);
adrNode FindNode(Graph G, infoGraph X);
void ResetVisited(Graph &G);

// Soal 2 dan 3
void PrintDFS(Graph G, adrNode N);
void PrintBFS(Graph G, adrNode N);

#endif

```

Main.cpp

```

#include "graph.h"
#include "graph.cpp"

int main() {
    Graph G;
    CreateGraph(G);

    // Tambah node sesuai Gambar pada soal
    InsertNode(G, 'A');
    InsertNode(G, 'B');
    InsertNode(G, 'C');
    InsertNode(G, 'D');
    InsertNode(G, 'E');
    InsertNode(G, 'F');
    InsertNode(G, 'G');
    InsertNode(G, 'H');

    // Hubungkan node sesuai Gambar pada soal
    adrNode A = FindNode(G, 'A');
    adrNode B = FindNode(G, 'B');
    adrNode C = FindNode(G, 'C');

```

```

    adrNode D = FindNode(G, 'D');
    adrNode E = FindNode(G, 'E');
    adrNode F = FindNode(G, 'F');
    adrNode GNode = FindNode(G, 'G');
    adrNode H = FindNode(G, 'H');

    // Koneksi sesuai gambar
    ConnectNode(A, B);
    ConnectNode(A, C);
    ConnectNode(B, D);
    ConnectNode(B, E);
    ConnectNode(C, F);
    ConnectNode(C, GNode);
    ConnectNode(D, H);
    ConnectNode(E, H);
    ConnectNode(F, H);
    ConnectNode(GNode, H);

    // Tampilkan struktur graph
    cout << "=== Struktur Graph ===" << endl;
    PrintInfoGraph(G);

    // Soal 2: DFS
    cout << "\n=== DFS dari Node A ===" << endl;
    ResetVisited(G);
    PrintDFS(G, A);
    cout << endl;

    // Soal 3: BFS
    cout << "\n=== BFS dari Node A ===" << endl;
    ResetVisited(G);
    PrintBFS(G, A);
    cout << endl;

    return 0;
}

```

Output

```
PS C:\Users\Lenovo\Documents\STRUKTUR DATA MODUL 14> cd "c:\Users\Lenovo\Documents\STRUKTUR DATA MODU
L 14\UNGUIDED\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
=== Struktur Graph ===
H -> G F E D
G -> H C
F -> H C
E -> H B
D -> H B
C -> G F A
B -> E D A
A -> C B

=== DFS dari Node A ===
A C G H F E B D

=== BFS dari Node A ===
A C B G F E D H
PS C:\Users\Lenovo\Documents\STRUKTUR DATA MODUL 14\UNGUIDED>
```

Deskripsi:

Program ini merupakan implementasi graf tidak berarah menggunakan adjacency list dalam C++ yang menyediakan fungsi untuk membuat graf, menambahkan node, menghubungkan node, serta melakukan traversal Depth-First Search (DFS) dan Breadth-First Search (BFS). Pada program utama, dibuat graf dengan delapan node ('A' hingga 'H') yang dihubungkan sesuai struktur tertentu, kemudian menampilkan representasi adjacency list-nya serta hasil traversal DFS dan BFS yang dimulai dari node 'A' untuk menunjukkan perbedaan urutan kunjungan antara kedua algoritma tersebut.

C. Kesimpulan

Modul ini dapat mengimplementasikan struktur data graf menggunakan adjacency list dalam C++ dengan fungsi dasar untuk pembuatan, penambahan node, penghubungan node, serta algoritma traversal DFS dan BFS. Implementasi menunjukkan perbedaan urutan kunjungan antara DFS (yang mendalami setiap cabang) dan BFS (yang menjelajahi node per tingkat), sesuai dengan teori graf, sekaligus memvalidasi bahwa representasi adjacency list efektif untuk memodelkan graf tidak berarah.

D. Referensi

<https://www.geeksforgeeks.org/cpp/implementation-of-graph-in-cpp/>

<https://medium.com/@itsvinayyay/graphs-data-structure-in-c-ab7b4205f41a>